TP n°2 – SSAD

Manipuler OpenSSL sous Ubuntu : correction pas à pas, pour débutant complet

M1 BigData – USTHB – Prof. H. Nacer – 2026/2027
Tout est expliqué en français, sans supposer que tu connais Linux. Les commandes ont été testées avec OpenSSL 3.0 (celui d'Ubuntu 22.04 / 24.04).

> **Comment lire ce document :** les rectangles noirs sont des commandes à taper (ou copier-coller) dans le terminal. On ne tape jamais le signe **$** : c'est juste le symbole qui indique que le terminal attend une commande. Appuie sur **Entrée** après chaque commande.

0. Linux en 5 minutes

Ouvrir le terminal

Sur Ubuntu, appuie sur **Ctrl + Alt + T**. Une fenêtre noire s'ouvre : c'est le terminal (le « shell »). Tu y tapes des commandes, il répond.

Les 8 commandes dont tu as besoin

```
pwd                     # affiche le dossier où tu es
ls                      # liste les fichiers du dossier
ls -l                   # pareil, avec la taille et les détails
mkdir tp2               # crée un dossier nommé tp2
cd tp2                  # entre dans le dossier tp2
cd ..                   # remonte d'un dossier
cat fichier.txt         # affiche le contenu d'un fichier
nano fichier.txt        # éditeur de texte (Ctrl+O puis Entrée = enregistrer,
                        #                   Ctrl+X = quitter)
```

Le texte après **#** est un commentaire : tu peux ne pas le taper. Astuce : la touche **↑** rappelle la commande précédente, et **Tab** complète les noms de fichiers.

Vérifier qu'OpenSSL est installé

```
openssl version
```

Résultat attendu : une ligne comme `OpenSSL 3.0.13 30 Jan 2024`. Si tu as « command not found », installe-le :

```
sudo apt update
sudo apt install openssl
```

À savoir : sudo veut dire « exécuter en administrateur ». Il demande ton mot de passe Ubuntu ; quand tu le tapes, **rien ne s'affiche** (même pas des étoiles) : c'est normal, tape-le et appuie sur Entrée.

1. C'est quoi OpenSSL ?

OpenSSL est une « boîte à outils » de cryptographie. Elle sait chiffrer des fichiers, générer des clés, calculer des empreintes (SHA, MD5), créer des certificats, etc. Ce TP utilise seulement la partie **chiffrement symétrique** : on chiffre et on déchiffre avec **la même clé** (un mot de passe secret).

Une commande OpenSSL s'écrit toujours : `openssl <commande> <options>`. Ici la commande est **enc** (encrypt = chiffrer).

2. Préparer le dossier de travail

```
mkdir ~/tp2
cd ~/tp2
echo "Bonjour, ceci est mon message secret" > mestest.txt
cat mestest.txt
```

~ désigne ton dossier personnel. **echo "..." > fichier** écrit le texte dans un fichier : tu viens de créer **mestest.txt**, le message en clair (le « plaintext »). Tu peux aussi le créer avec `nano mestest.txt`.

3. Obtenir de l'aide (partie « What is OpenSSL »)

```
openssl list -cipher-commands
openssl enc -help
man openssl
```

- **openssl list -cipher-commands** : liste tous les algorithmes de chiffrement disponibles (aes-256-cbc, aes-128-cbc, aria...).
- **openssl enc -help** : explique toutes les options de la commande enc (le « Enc -help » de l'énoncé).
- **man openssl** : le manuel complet ; quitte avec la touche **q**.

À savoir : l'énoncé écrit `openssl list-cipher-commands`. Avec OpenSSL 3, cette forme donne « Invalid command ». La bonne écriture est `openssl list -cipher-commands` (avec un espace et un tiret).

Le mode interactif

```
openssl
OpenSSL> version
OpenSSL> q
```

Taper seulement **openssl** ouvre l'invite **OpenSSL>**. Tu y tapes des commandes sans écrire « openssl » devant. **q** (ou Ctrl+D) pour en sortir et revenir au **$**.

4. Générer la clé privée (le mot de passe secret)

La clé est ce qui rend le chiffrement sûr : sans elle, impossible de lire le message. On la stocke dans un fichier.

Cas 1 : avec openssl rand

```
openssl rand -out cle.key 2048
ls -l cle.key
```

Génère 2048 octets aléatoires dans le fichier **cle.key**. Dans `rand`, **-out** = fichier de sortie, et le nombre = taille.

À savoir : avec **rand**, le nombre est en **octets**, pas en bits (le cours dit « bits » mais la commande donne 2048 octets, vérifie avec `ls -l`). Le fichier est du binaire illisible : si tu fais `cat cle.key`, tu verras des symboles bizarres, c'est normal. Pour avoir une clé lisible, ajoute **-base64** :

```
openssl rand -base64 32 > mdp.key
cat mdp.key
```

Cas 2 : avec openssl genrsa (clé RSA)

```
openssl genrsa -out cle.key 1024
head -2 cle.key
```

Génère une clé privée RSA de 1024 bits dans **cle.key**. Ouvre-la : tu vois `-----BEGIN PRIVATE KEY-----` puis du texte. (Ici le nombre est bien en bits.)

À savoir : la commande `rand` et `genrsa` écrivent dans le même nom **cle.key** : la deuxième **écrase** la première. Garde un seul des deux pour la suite, ou utilise deux noms différents (ex. **cle_rand.key** et **cle_rsa.key**). Une clé RSA de 1024 bits est considérée comme faible aujourd'hui (on utilise 2048 ou plus) ; ici c'est juste pour le TP.

5. Chiffrer un fichier

Forme générale (celle de l'énoncé) :

```
openssl enc -<algo> -in <entree> -out <sortie> -k <cle>
```

- **enc** : commande de chiffrement symétrique.
- **-algo** : l'algorithme choisi, ex. **-aes-256-cbc**.
- **-in** : le fichier à chiffrer (entrée, ici mestest.txt).
- **-out** : le fichier qui recevra le résultat chiffré.
- **-k** ou **-kfile** : indique la clé (voir l'encadré ci-dessous).

- **-base64** : écrit le résultat en texte lisible au lieu de binaire (pratique pour l'afficher ou l'envoyer par message).

Exemple complet

```
openssl enc -aes-256-cbc -pbkdf2 -base64 -in mestest.txt \
    -out crypted -kfile cle.key
cat crypted
```

Résultat attendu : le contenu de **crypted** est un texte incompréhensible qui commence par `U2FsdGVkX1...`. C'est ton message chiffré.

-k ou -kfile ? (important)
• **-k texte** : le « texte » qui suit est utilisé **directement comme mot de passe**. Si tu écris `-k cle.key`, le mot de passe est le mot « cle.key » (pas le contenu du fichier !). Ça marche quand même, mais ce n'est pas ce qu'on veut.
• **-kfile fichier** : OpenSSL **lit le mot de passe dans le fichier**. C'est le bon choix pour utiliser cle.key comme clé. (Pour une clé binaire de `rand`, préfère `-kfile mdp.key` avec la version base64.)
• Sans rien : OpenSSL te demande le mot de passe au clavier.

À savoir : tu verras peut-être « *** **WARNING : deprecated key derivation used** ». Ce n'est pas une erreur : OpenSSL te conseille d'ajouter **-pbkdf2** (méthode moderne de dérivation de clé). Ajoute-le toujours, au chiffrement **et** au déchiffrement. Si tu l'oublies d'un côté seulement, le déchiffrement échoue.

À savoir : l'énoncé cite DES. Avec OpenSSL 3, **DES est désactivé** par défaut (erreur « unsupported »). Utilise AES : **aes-128-cbc** ou **aes-256-cbc**.

6. Déchiffrer

C'est la même commande avec **-d** (decrypt), en inversant entrée et sortie :

```
openssl enc -aes-256-cbc -pbkdf2 -base64 -d -in crypted \
    -out retour.txt -kfile cle.key
cat retour.txt
```

Résultat attendu : `Bonjour, ceci est mon message secret` : tu retrouves ton message d'origine. Vérifie qu'il est identique avec : `diff mestest.txt retour.txt` (aucun affichage = identiques).

Les exemples de l'énoncé, corrigés

```
# énoncé : $ enc-aes -in mestest.txt -d -out mescrypt.txt -k cle.key
# (c'est un raccourci du cours, la vraie commande est) :
openssl enc -aes-256-cbc -pbkdf2 -in mestest.txt \
    -out mescrypt.txt -kfile cle.key
openssl enc -aes-256-cbc -pbkdf2 -d -in mescrypt.txt \
    -out mestest_dec.txt -kfile cle.key

# énoncé : openssl enc -aes-256-cbc-base64 -in output.txt -d -out messresultat.tx
openssl enc -aes-256-cbc -pbkdf2 -base64 -d -in crypted \
    -out messresultat.txt -kfile cle.key
```

À savoir : dans l'énoncé il y a des fautes de frappe : **aes-256-cc** doit être **aes-256-cbc**, **aes-256-cbc-base64** doit s'écrire **-aes-256-cbc -base64** (deux options), et « `Openssl –out key 1024` » doit être `openssl genrsa -out key 1024`. Aussi, les tirets copiés depuis le PDF (–) sont parfois de faux tirets : retape-les au clavier avec la touche **-**.

Résumé « Chiffrer / Déchiffrer »

```
# générer la clé
openssl rand -base64 32 > cle.key
# chiffrer
openssl enc -aes-256-cbc -pbkdf2 -base64 -in message.txt \
    -out crypted -kfile cle.key
# lire le chiffré
cat crypted
# déchiffrer
openssl enc -aes-256-cbc -pbkdf2 -base64 -d -in crypted \
    -out retourmess.txt -kfile cle.key
```

7. Les 3 cas d'utilisation du TP

L'idée : **Alice** chiffre un message, l'envoie à **Bob**, qui le déchiffre avec la même clé. Le problème central du chiffrement symétrique : **la clé doit être transmise à Bob de façon sûre** (jamais dans le même message que le fichier chiffré).

Cas 1 : une seule machine, sans machine virtuelle

Tu joues Alice et Bob toi-même avec deux dossiers.

```
mkdir -p ~/tp2/alice ~/tp2/bob
cd ~/tp2/alice
echo "Salut Bob, rendez-vous a 10h" > message.txt
openssl rand -base64 32 > cle.key
openssl enc -aes-256-cbc -pbkdf2 -base64 -in message.txt \
    -out crypted -kfile cle.key

# "envoi" : on copie dans le dossier de Bob
cp crypted cle.key ../bob/

cd ../bob
openssl enc -aes-256-cbc -pbkdf2 -base64 -d -in crypted \
    -out message_recu.txt -kfile cle.key
cat message_recu.txt
```

Résultat attendu : `Salut Bob, rendez-vous a 10h.`

Test d'échec utile pour le rapport : change une lettre dans **crypted** (avec nano) ou utilise une autre clé, puis déchiffre : tu obtiens bad decrypt ou un résultat illisible.

Cas 2 : une machine physique avec deux machines virtuelles

Tu n'as pas de machine virtuelle : il y a deux solutions. Les deux marchent avec les mêmes commandes openssl.

Solution A (simple et rapide) : simuler deux machines avec deux utilisateurs Ubuntu.

```
sudo adduser bob          # crée l'utilisateur bob (mets un mot de passe,
                          # le reste : Entrée)
mkdir -p /tmp/echange
chmod 777 /tmp/echange    # dossier partagé entre alice (toi) et bob

# --- Terminal 1 (toi = Alice) ---
cd ~/tp2/alice
openssl enc -aes-256-cbc -pbkdf2 -base64 -in message.txt \
    -out crypted -kfile cle.key
cp crypted cle.key /tmp/echange/
chmod 644 /tmp/echange/*

# --- Terminal 2 : ouvre un 2e terminal (Ctrl+Alt+T) ---
su - bob                  # tu deviens Bob (taper son mot de passe)
cd /tmp/echange
openssl enc -aes-256-cbc -pbkdf2 -base64 -d -in crypted \
    -out recu.txt -kfile cle.key
cat recu.txt
exit                      # retour à ton utilisateur
```

Solution B (vraies machines virtuelles) : VirtualBox.

```
sudo apt install virtualbox
```

Télécharge l'image ISO d'Ubuntu (ubuntu.com/download/desktop), crée 2 machines virtuelles « VM-Alice » et « VM-Bob » (2 Go de RAM, 20 Go de disque chacune) et installe Ubuntu dedans. Dans les **Paramètres > Réseau** de chaque VM, choisis **« Accès par pont »** pour qu'elles se voient. Ensuite, dans les deux VM, installe le serveur SSH et relève l'adresse IP :

```
sudo apt install openssh-server
ip a                      # cherche la ligne "inet 192.168.x.x"
```

Puis, depuis la VM-Alice, envoie les fichiers à la VM-Bob (voir cas 3 ci-dessous pour **scp**). Si ton PC est lent, la solution A suffit à montrer le principe ; précise simplement dans ton rapport que tu as simulé les deux machines.

Cas 3 : deux machines physiques

Il te faut deux ordinateurs sur le même réseau (même Wi-Fi/box), par exemple ton PC et celui d'un camarade. Sur **les deux** : `sudo apt install openssh-server`. Sur celui de Bob, trouve l'IP avec

`ip a` (ex. 192.168.1.25) et son nom d'utilisateur avec `whoami`.

```
# --- Sur la machine d'Alice ---
openssl enc -aes-256-cbc -pbkdf2 -base64 -in message.txt \
    -out crypted -kfile cle.key

# envoi du fichier chiffré vers Bob (remplace bob et l'IP) :
scp crypted bob@192.168.1.25:~/
# envoi de la clé, de préférence par un AUTRE canal :
scp cle.key bob@192.168.1.25:~/

# --- Sur la machine de Bob ---
openssl enc -aes-256-cbc -pbkdf2 -base64 -d -in crypted \
    -out recu.txt -kfile cle.key
cat recu.txt
```

À savoir : scp copie un fichier d'une machine à l'autre ; il demande le mot de passe de Bob. Pas de 2e PC sous la main ? Tu peux aussi échanger les fichiers par **clé USB**, ou envoyer **crypted** par e-mail/WhatsApp (c'est du texte avec -base64) et la clé par un autre moyen. Pour t'entraîner sur ton seul PC : `scp crypted $USER@localhost:~/` (nécessite openssh-server).

À savoir : bonne pratique de sécurité à écrire dans ton rapport : la clé ne doit **jamais** voyager avec le message chiffré par le même canal, sinon n'importe qui qui intercepte les deux peut déchiffrer. C'est la limite du chiffrement symétrique, que la cryptographie asymétrique (RSA) résout.

8. Dépannage

Message d'erreur	Cause et solution
command not found	OpenSSL absent : sudo apt install openssl. Ou faute de frappe.
No such file or directory	Tu n'es pas dans le bon dossier : tape pwd et ls, puis cd.
bad decrypt	Mauvaise clé, fichier modifié, ou -pbkdf2 / -base64 / algo différent entre chiffrement et déchiffrement.

Error setting cipher DES	DES désactivé dans OpenSSL 3 : utilise aes-256-cbc.
Invalid command list-cipher-commands	Écris : openssl list -cipher-commands
Permission denied	Droits manquants : vérifie avec ls -l, ou ajoute sudo.
deprecated key derivation	Juste un avertissement : ajoute -pbkdf2.

9. Pour ton rapport

- Pour chaque cas (1, 2, 3) : copie les commandes tapées et fais une **capture d'écran** du terminal avec le message chiffré (`cat crypted`) et le message retrouvé.
- Explique chaque option : **enc, -aes-256-cbc, -in, -out, -d, -k/-kfile, -base64, -pbkdf2**.
- Ajoute un test d'échec (mauvaise clé = « bad decrypt ») et la remarque sur l'échange sécurisé de la clé.
- Compare rapidement : **rand** (clé aléatoire, taille en octets) et **genrsa** (clé RSA, taille en bits).

Remarque : les commandes ont été exécutées et vérifiées avec OpenSSL 3.0.13 ; les cas 2 (VM) et 3 (deux PC) sont décrits mais n'ont pas pu être testés réseau en direct ici.